Leave Management System (LMS) - Architecture & Flow Guide

1. Database Schema & ERD (Entity Relationship Diagram)

The database is built on PostgreSQL (previously SQLite) using SQLAlchemy ORM.

Tables and Constraints

- **employees**:
  - *Primary Key*: `employee_id`
  - *Unique Constraints*: `employee_code`, `email`
  - *Foreign Keys*: `manager_id` -> `employees.employee_id`
- **leave_types**:
  - *Primary Key*: `leave_type_id`
  - *Unique Constraints*: `leave_code`
- **leave_balances**:
  - *Primary Key*: `balance_id`
  - *Foreign Keys*: `employee_id` -> `employees`, `leave_type_id` -> `leave_types`
- **leave_requests**:
  - *Primary Key*: `request_id`
  - *Foreign Keys*: `employee_id` -> `employees`, `leave_type_id` -> `leave_types`, `delegated_to_id` -> `employees`
- **approval_history**:
  - *Primary Key*: `history_id`
  - *Foreign Keys*: `request_id` -> `leave_requests`
- **audit_logs**:
  - *Primary Key*: `id`
- **notifications**:
  - *Primary Key*: `id`
  - *Foreign Keys*: `employee_id` -> `employees`

2. Authentication: JWT & CORS

- **JWT (JSON Web Tokens)**:
  - Tokens are created in the `/auth/login` endpoint after verifying the password via `bcrypt`.
  - The JWT payload uses the `sub` claim to store the `employee_id` and `exp` for the expiration timestamp (default: 24 hours).
  - Encoded using `HS256` and a secure SECRET_KEY.
  - The backend guards routes using the `get_current_user` dependency (FastAPI `Depends(oauth2_scheme)`), validating the bearer token on each request.
- **CORS (Cross-Origin Resource Sharing)**:
  - Handled in `main.py` using FastAPI's `CORSMiddleware`.
  - Currently allows all origins (`["*"]`), all headers, and all methods. This allows the Angular frontend (`http://localhost:4200`) to communicate freely with the backend.

3. Error Handling

- **Backend**: Uses FastAPI's `HTTPException`. For instance, invalid credentials or expired tokens trigger a `401 Unauthorized` error. Business logic errors (e.g., insufficient leave balance) trigger a `400 Bad Request`.
- **Frontend**: Angular intercepts HTTP errors globally or handles them in services (e.g., `auth.service.ts`) to display user-friendly Toast notifications.

4. Application Flow & Routing

Frontend Routing (Angular)

Defined in `app.routes.ts`:

- `/login`: Public login page.
- `/dashboard`: Protected landing page (requires `authGuard`).
- `/leave-request`: Form for applying for leaves.
- `/manage-balances` (HR): Leave balance management.
- `/audit-logs` (Admin): System logs.

Backend Routing (FastAPI)

Defined in `app/api/router.py`:

- `/api/v1/auth/*`: Login and token generation.
- `/api/v1/users/*`: Profile and user management.
- `/api/v1/leaves/*`: Applying, approving, or listing leaves.
- `/api/v1/notifications/*`: Fetching and marking notifications as read.
- `/api/v1/reports/*`: HR reporting.
- `/api/v1/audit-logs/*`: System tracking.

5. Flow Example: Applying for a Leave

Here is the step-by-step trace of how a leave application moves from the UI to the Database.

1. Frontend Component (`LeaveRequestComponent`): User clicks "Submit". The component gathers the form data and calls the service.

```
// Angular Service (leave.service.ts)
applyLeave(leaveData: any): Observable<any> {
  // The token is automatically attached by an HTTP Interceptor or passed manually in headers
  return this.http.post('http://localhost:8000/api/v1/leaves/apply', leaveData);
}
```

2. Backend API Endpoint (`app/api/endpoints/leaves.py`): The request hits FastAPI. FastAPI first runs the `get_current_user` dependency to validate the JWT. If valid, the route executes.

```python
@router.post("/apply")
def apply_for_leave(
    request_data: LeaveApplySchema,
    db: Session = Depends(get_db),
    current_user: Employee = Depends(get_current_user)
):
    # 3. Business Logic Execution
    # ... checks leave balance for current_user ...

    # 4. Database Insert
    new_request = LeaveRequest(
        employee_id=current_user.employee_id,
        leave_type_id=request_data.leave_type_id,
        start_date=request_data.start_date,
        end_date=request_data.end_date,
        reason=request_data.reason,
        status=LeaveStatus.PENDING.value
    )
    db.add(new_request)
    db.commit()
    db.refresh(new_request)

    return {"message": "Leave applied successfully", "data": new_request}
```

5. Database Transaction: SQLAlchemy generates the `INSERT INTO leave_requests ...` SQL command and commits the transaction to the PostgreSQL database. The response is sent back to Angular with a `200 OK` status, and the user is redirected to their dashboard.